

Cole Nola - Assignment 2

Analysis

2a.

The two GenAI programs used for this assignment were ChatGPT and Google Gemini.

2b.

I accessed both Chatgpt and Gemini through google browser

2c.

The same assignment prompt was used for both GenAI programs:

Write in c:

The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.
- Test File format:

Command Description

EMAIL <sender category>,

<subject line>,

<date>

The emails in the CEO's Inbox should be placed in queue

based on their sender category and date. The sender

categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be

able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

2d.

ChatGPT-Generated Code (unmodified)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000

typedef struct {
    char sender[30];
    char subject[200];
    char date[11];
    int priority;
} Email;

typedef struct {
    Email heap[MAX_EMAILS];
    int size;
} MaxHeap;

/* Get priority based on sender category */
int getPriority(char sender[]) {
    if (strcmp(sender, "Boss") == 0)
        return 5;
    else if (strcmp(sender, "Subordinate") == 0)
        return 4;
    else if (strcmp(sender, "Peer") == 0)
        return 3;
    else if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    else
        return 1;
}

/* Convert date to a number so newer dates are larger */
int dateValue(char date[]) {
    int month, day, year;

    sscanf(date, "%d-%d-%d", &month, &day, &year);

    return year * 10000 + month * 100 + day;
}

/* Determine if email a has higher priority than email b */
int higherPriority(Email a, Email b) {
    if (a.priority != b.priority)
        return a.priority > b.priority;

    return dateValue(a.date) > dateValue(b.date);
}

/* Swap two emails */
void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* Move an email upward in the heap */
void heapifyUp(MaxHeap *heap, int index) {
    int parent;

    while (index > 0) {
        parent = (index - 1) / 2;
```

```

    if (higherPriority(heap->heap[index], heap->heap[parent])) {
        swap(&heap->heap[index], &heap->heap[parent]); index = parent;
    } else {
        break;
    }
}

/* Move an email downward in the heap */
void heapifyDown(MaxHeap *heap, int index) {
    int left, right, largest;

    while (1) {
        left = 2 * index + 1;
        right = 2 * index + 2;
        largest = index;

        if (left < heap->size &&
            higherPriority(heap->heap[left], heap->heap[largest])) { largest =
left;
        }

        if (right < heap->size &&
            higherPriority(heap->heap[right], heap->heap[largest])) { largest =
right;
        }

        if (largest != index) {
            swap(&heap->heap[index], &heap->heap[largest]); index =
largest;
        } else {
            break;
        }
    }

    /* Add an email to the MaxHeap */
    void insert(MaxHeap *heap, Email email) {
        if (heap->size >= MAX_EMAILS)
            return;

        heap->heap[heap->size] = email;
        heapifyUp(heap, heap->size);
        heap->size++;
    }

    /* Remove the highest-priority email */
    Email removeMax(MaxHeap *heap) {
        Email removed = heap->heap[0];

        heap->heap[0] = heap->heap[heap->size - 1];
        heap->size--;

        if (heap->size > 0)
            heapifyDown(heap, 0);

        return removed;
    }

    /* Display the highest-priority email without removing it
    */ void nextEmail(MaxHeap *heap) {
        if (heap->size == 0)
            return;

        printf("Next email:\n");
        printf("Sender: %s\n", heap->heap[0].sender);
        printf("Subject: %s\n", heap->heap[0].subject);
        printf("Date: %s\n", heap->heap[0].date);
    }

    int main(int argc, char *argv[]) {
        MaxHeap heap;
        heap.size = 0;

        FILE *file;

```

```

/* Open the test file */
if (argc > 1)
file = fopen(argv[1], "r");
else
file = stdin;

if (file == NULL) {
printf("Could not open file.\n");
return 1;
}

char line[300];

while (fgets(line, sizeof(line), file) != NULL) {

/* Remove newline */
line[strcspn(line, "\n")] = '\0';

/* EMAIL command */
if (strncmp(line, "EMAIL ", 6) == 0) {
Email email;

char *data = line + 6;

char *sender = strtok(data, ",");
char *subject = strtok(NULL, ",");
char *date = strtok(NULL, ",");

if (sender != NULL && subject != NULL && date != NULL) {
strcpy(email.sender, sender);
strcpy(email.subject, subject);
strcpy(email.date, date);

email.priority = getPriority(email.sender);

insert(&heap, email);
}
}

/* NEXT command */
else if (strcmp(line, "NEXT") == 0) {
nextEmail(&heap);
}

/* READ command */
else if (strcmp(line, "READ") == 0) {
if (heap.size > 0)
removeMax(&heap);
}

/* COUNT command */
else if (strcmp(line, "COUNT") == 0) {
printf("There are %d emails to read.\n", heap.size);
}

if (file != stdin)
fclose(file);

return 0;
}

```

Gemini-Generated Code (unmodified)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINE 256

typedef struct {
char sender[32];

```

```

char subject[128];
char date_str[11];
int priority_score;
int date_key;
} Email;

typedef struct {
    Email *data;
    int size;
    int capacity;
} MaxHeap;

// Maps sender category to numerical priority
int get_sender_priority(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
    if (strcmp(sender, "Subordinate") == 0) return 4;
    if (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2;
    if (strcmp(sender, "OtherPerson") == 0) return 1;
    return 0;
}

// Converts MM-DD-YYYY into YYYYMMDD integer for easy chronological
comparison int parse_date_key(const char *date_str) {
    int month, day, year;
    sscanf(date_str, "%d-%d-%d", &month, &day, &year);
    return year * 10000 + month * 100 + day;
}

// Returns > 0 if email 'a' has higher priority than email
'b' int compare_emails(Email a, Email b) {
    if (a.priority_score != b.priority_score) {
        return a.priority_score - b.priority_score;
    }
    // Newer date takes precedence
    return a.date_key - b.date_key;
}

MaxHeap* create_heap(int initial_capacity) {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->capacity = initial_capacity;
    heap->size = 0;
    heap->data = (Email*)malloc(sizeof(Email) * heap->capacity);
    return heap;
}

void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

void heapify_up(MaxHeap *heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare_emails(heap->data[index], heap->data[parent]) > 0) {
            swap(&heap->data[index], &heap->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

void heapify_down(MaxHeap *heap, int index) {
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < heap->size && compare_emails(heap->data[left], heap->data[largest]) > 0) {
            largest = left;
        }
        if (right < heap->size && compare_emails(heap->data[right], heap->data[largest]) > 0) {
            largest = right;
        }
    }
}

```

```

    }

    if (largest != index) {
        swap(&heap->data[index], &heap->data[largest]);
        index = largest;
    } else {
        break;
    }
}

void insert_email(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->data = (Email*)realloc(heap->data, sizeof(Email) * heap->capacity);
    }
    heap->data[heap->size] = email;
    heapify_up(heap, heap->size);
    heap->size++;
}

void pop_max(MaxHeap *heap) {
    if (heap->size == 0) return;
    heap->data[0] = heap->data[heap->size - 1];
    heap->size--;
    heapify_down(heap, 0);
}

void free_heap(MaxHeap *heap) {
    free(heap->data);
    free(heap);
}

int main(int argc, char *argv[]) {
    MaxHeap *heap = create_heap(10);
    char line[MAX_LINE];
    FILE *file = stdin;

    // Open sample.txt if passed via command line argument
    if (argc > 1) {
        file = fopen(argv[1], "r");
        if (!file) {
            fprintf(stderr, "Error opening file: %s\n", argv[1]);
            free_heap(heap);
            return 1;
        }
    }

    while (fgets(line, sizeof(line), file)) {
        // Strip trailing newline characters
        line[strcspn(line, "\r\n")] = 0;

        if (strncmp(line, "EMAIL ", 6) == 0) {
            Email e;
            char *payload = line + 6;

            // Parse comma-delimited string fields
            char *token = strtok(payload, ",");
            if (token) strncpy(e.sender, token, sizeof(e.sender) - 1);
            token = strtok(NULL, ",");
            if (token) strncpy(e.subject, token, sizeof(e.subject) - 1);
            token = strtok(NULL, ",");
            if (token) strncpy(e.date_str, token, sizeof(e.date_str) - 1);

            e.priority_score = get_sender_priority(e.sender);
            e.date_key = parse_date_key(e.date_str);

            insert_email(heap, e);
        }
        else if (strcmp(line, "COUNT") == 0) {
            printf("There are %d emails to read.\n", heap->size);
        }
        else if (strcmp(line, "NEXT") == 0) {
            if (heap->size > 0) {
                Email top = heap->data[0];
                printf("Next email:\n");
                printf("Sender: %s\n", top.sender);
            }
        }
    }
}

```



```

printf("Subject: %s\n", top.subject);
printf("Date: %s\n", top.date_str);
}
}
else if (strcmp(line, "READ") == 0) {
if (heap->size > 0) {
pop_max(heap);
}
}
}

if (file != stdin) {
fclose(file);
}

free_heap(heap);
return 0;
}

```

2e.

Category	Chatgpt	Gemini
Correctness	Correct for well-formed input; reproduces the assignment's sample output exactly. It compares sender priority, then date, and stops/starts correctly on NEXT, READ, and COUNT. It copies each field with strcpy(), which does not check length, so an unusually long sender/subject/date value in a test file could overflow the fixed-size struct fields.	Correct for well-formed input; also reproduces the sample output exactly and handles the same command set correctly. It copies fields with strncpy() instead of strcpy(), which is safer, but it does not explicitly null-terminate the destination afterward, so an especially long field could still leave the string unterminated.
Execution Time	$O(\log n)$ per insert or remove, because both call heapifyUp/heapifyDown; $O(1)$ for NEXT and COUNT.	Same $O(\log n)$ insert/remove via heapify_up/heapify_down and $O(1)$ NEXT/COUNT; identical asymptotic behavior to the ChatGPT version.
Space Complexity	Fixed at compile time: the heap always reserves room for 1000 emails (Email heap[1000]) no matter how many are actually used, and cannot hold more than 1000 emails at all.	$O(n)$: starts with room for only 10 emails and doubles the array with realloc() as needed, so memory use grows with the actual input and there is no hard cap on the number of emails.
Maintainability	Simple and easy to follow; no dynamic memory management, so there is no risk of a memory leak. Already had short one-line comments above each function, but no prologue and no line-by-line comments.	Slightly more defensive sender-category handling (an unrecognized category is scored separately from OtherPerson), but the added malloc/realloc/free bookkeeping creates more places to introduce a bug, and the draft had almost no comments.

2f.

Chatgpt was selected. Both programs satisfy the core assignment for valid input, but the ChatGPT version has stronger maintainability at the starting point: it needs no dynamic memory management, which removes the risk of a memory leak or a bad `realloc()`, and it was already closer to meeting the comment rubric. Its two real weaknesses next to the Gemini draft - the unchecked `strcpy()` calls and the fixed 1000-email limit - are narrow, well-understood problems that can be fixed directly (bounds-checked copying, and a dynamically-growing array) without inheriting Gemini's missing-null-terminator risk or its larger memory-management surface area.

2g.

The Chatgpt program was improved in the following ways:

- **Correctness:** every `strcpy()` call was replaced with a new `safeCopy()` helper that uses `strncpy()` and then always writes an explicit null terminator, so an over-long sender, subject, or date field can no longer overflow the struct or leave a string unterminated.
- **Execution time:** no algorithmic change was needed. insert and remove remain $O(\log n)$ via `heapifyUp/heapifyDown`, which was already correct.
- **Space complexity:** the fixed Email heap[1000] array was replaced with a dynamically-allocated array (`malloc` at start, `realloc` to double the capacity whenever it fills up), adapted from the idea behind Gemini's draft. The program now uses memory proportional to the number of emails actually inserted, with no hard cap.
- **Maintainability:** the final source includes a full prologue comment block (program name, description, inputs, outputs, collaborators, sources, author, dates, revisions), a block comment above every function, and line-by-line comments explaining each statement, to meet the assignment's comment rubric. NULL checks were also added after every `malloc()/realloc()` call, and an explicit `freeHeap()` function frees all heap memory before the program exits.